Credit Risk Monitor - Architecture

Raw data -> ingestion -> analytics -> model -> dashboard, with both AI features. Tool / library at each stage in brackets. Pipeline: python src/build.py | App: streamlit run app/app.py

1. Raw data (data/raw/)

5 CSVs, never modified:
customers (10,000)
loan_accounts (14,685, target)
bureau_credit (22,979)
repayment_history (176,220)
collections_events (8,635)
[CSV files in the GitHub repo]

2. Ingestion (src/ingest.py)

[pandas, numpy]
Clean: gender labels, group-median fill
+ missing flags, cap impossible values
Summarize repayment: 12 rows -> 1 per loan
Summarize bureau: n lines -> 1 per customer
Left joins on loans, validate= checks,
assert 14,685 rows
Data-quality log of every decision

3. Clean dataset

data/processed/
analytic_dataset.csv
1 row per loan, 65 columns
data_quality_log.csv

Collections columns kept for
dashboard only (not model input)

AI 1: Ask the Portfolio

src/llm_assistant.py

1. Planner call: question + tool list -> JSON {tool, args} or decline
2. Code validates the tool; pandas computes numbers (analytics.py functions)
3. Writer call: question + small result table -> 1-3 sentence answer
4. Code checks EVERY number vs the table; mismatch -> code-written safe answer
5. No key / API down -> quick questions, no crash

(runs in the Ask tab)

1

2

Google Gemini API

[google-genai SDK]
gemini-3.5-flash,
fallbacks: flash-lite,
flash-latest

Key: Streamlit Secrets
(local: .env, git-ignored)
temperature = 0

4a. Analytics (src/analytics.py)

[pandas]
KPIs, profile by dimension,
cross-cuts, 5 insights with 'so what'
-> insights.json

One source of truth: the dashboard charts AND the LLM tools call these same functions.

4b. Default model (src/model.py)

[scikit-learn, XGBoost]
Logistic regression baseline + XGBoost
Split by customer (75/25), early stopping
AUC / KS / Gini, lift, calibration
TreeSHAP drivers per account
No IDs, dates, gender, marital, collections
-> scored_accounts.csv, models/*.joblib

4c. Anomaly detection

src/anomaly.py [pandas, sklearn]
Own history: sudden miss,
DPD jump (months 1-9 vs 10-12)
Peer: top 1% bureau utilization /
DPD in loan type x income band
Isolation Forest: rarest 2%
-> anomalies.csv + reasons

Where the LLM is called and what is sent

Call 1 (planner) sends:
the analyst's question + the list of approved tools.
NO data at all.
Call 2 (writer) sends:
the question + one small aggregated result table
(e.g. default rate by loan type, or one account's
PD + drivers). NO raw customer rows.
Never sent: raw CSVs, the full dataset, the API key.
The LLM never calculates; Python does.

5. Dashboard (app/app.py + app/views/) [Streamlit, Plotly] - on Streamlit Cloud

Sidebar filters apply to every tab.
Portfolio: KPIs, risk segments, exposure, profile, cross-cut heatmap, trends, drill-down, insights
Account: PD (both models), SHAP key drivers, 12-month repayment chart, bureau, other loans, flags
Ask the Portfolio (AI feature 1): question -> tool -> verified answer + source table, or decline
Anomaly Review (AI feature 2): review list, reasons, Confirm / Dismiss + note -> decision log
Collections: action pay rates with confidence bars, cost calculator, recommended strategy
Model & Monitoring: metrics, lift, calibration, drivers, PSI drift check, monitoring plan, fairness
Every tab states trigger / false positive / false negative / human step for its AI feature.

Human-in-the-loop

Risk analyst
Ask: checks the Verified badge + source table
before using a number.
Anomalies: reads the reason, records
Confirm / Dismiss + note.
-> data/decisions/decisions.csv
(who, when, why; CSV download)
Demo storage resets on cloud restart;
production = database + audit trail.
The system never acts on its own.

Repo: github.com/romitsoni/credit-risk-dashboard | App: credit-risk-monitor-romit.streamlit.app | Python 3.12, packages pinned in requirements.txt